

Deep Learning Assignment – I

Submitted by

Anushaa B (Roll No.: CS24BT032)

Skanda P (Roll No.: CS24BT045)

Thiaana B (Roll No.: CS24BT031)

Under the Supervision of

Prof Dileep AD



Department of Computer Science and Engineering

Indian Institute of Technology Dharwad

August, 2026

Abstract

This assignment focuses on implementing a perceptron model from scratch for classification and regression. NumPy and Matplotlib are used for numerical operations and plots, but no external machine-learning library is used for the model, training, data splitting, evaluation, or experiments. For classification, the perceptron is tested on linearly separable and non-linearly separable data using logistic and tanh activation functions. Three-class classification is handled using one-against-one voting. For regression, a linear activation is applied to univariate and bivariate data. Different learning rates and epoch counts are tested, and the models are evaluated using learning curves, decision regions, confusion matrices, classification metrics, RMSE, and prediction plots. The linearly separable data reaches 100.00% accuracy, while the highest non-linearly separable accuracy is 82.67%. The best percentage RMSE values are 7.11% for univariate regression and 17.83% for bivariate regression. The results show that the perceptron performs well when the data follows a linear pattern, but its performance is limited when the data requires nonlinear decision boundaries.

Contents

1	Introduction	1
2	Proposed Methodology	2
2.1	Single-Neuron Model and Training	2
2.2	Activation Functions	3
2.3	Prediction and Evaluation	3
3	Experimental Setup	5
4	Results	6
4.1	Classification Results	6
4.1.1	Hyperparameter Comparison	6
4.1.2	Inferences from the Classification Sweep	7
4.1.3	Linearly Separable Dataset	8
4.1.4	Non-linearly Separable Dataset	12
4.1.5	Classification Comparison	16
4.2	Regression Results	16
4.2.1	Hyperparameter Comparison	16
4.2.2	Univariate Regression	18
4.2.3	Bivariate Regression	19
5	Conclusion	20
	References	21

List of Figures

4.1	Linearly separable logistic loss curves.	8
4.2	Linearly separable logistic decision regions.	9
4.3	Linearly separable tanh loss curves.	10
4.4	Linearly separable tanh decision regions.	11
4.5	Non-linearly separable logistic loss curves.	12
4.6	Non-linearly separable logistic decision regions.	13
4.7	Non-linearly separable tanh loss curves.	14
4.8	Non-linearly separable tanh decision regions.	15
4.9	Univariate regression plots for $LR = 0.05$, 1000 epochs.	18
4.10	Bivariate regression plots for $LR = 0.05$, 1000 epochs.	19

List of Tables

4.1	Classification test accuracy (%) for the complete sweep.	6
4.2	Accuracy range and number of tied best settings.	7
4.3	Linearly separable logistic confusion matrix and class-wise test metrics.	9
4.4	Linearly separable tanh confusion matrix and class-wise test metrics.	11
4.5	Non-linearly separable logistic confusion matrix and class-wise test metrics.	13
4.6	Non-linearly separable tanh confusion matrix and class-wise test metrics.	15
4.7	Regression results for every recorded setting.	16

Chapter 1

Introduction

This assignment implements a perceptron model from scratch for classification and regression. The classification tasks use linearly separable and non-linearly separable datasets with logistic and tanh activation functions. Three-class prediction is handled using one-against-one voting. The regression tasks use a linear activation function with univariate and bivariate data.

The work also includes data splitting, gradient-descent training, evaluation metrics, plots, and hyperparameter experiments. These components are written without using a machine-learning library. The experiments study how learning rate, epoch count, activation function, and data shape affect the results.

Chapter 2

Proposed Methodology

2.1 Single-Neuron Model and Training

For sample i , with feature vector $\mathbf{x}_i$, the neuron computes

$$z_i = \mathbf{w}^T \mathbf{x}_i + b, \quad \hat{y}_i = f(z_i), \quad (2.1)$$

where the weight vector $\mathbf{w}$ and bias b start at zero. The model is based on the perceptron and feed-forward neuron [1–3]. Training minimizes the half-MSE loss

$$\mathcal{L} = \frac{1}{2n} \sum_{i=1}^n (\hat{y}_i - y_i)^2. \quad (2.2)$$

Defining $\delta_i = (\hat{y}_i - y_i)f'(z_i)$, one full-batch update per epoch is

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{1}{n} \mathbf{X}^T \boldsymbol{\delta}, \quad b \leftarrow b - \eta \frac{1}{n} \sum_{i=1}^n \delta_i, \quad (2.3)$$

where η is the learning rate. The factor $1/2$ cancels the factor 2 from the derivative. The learning curves show the MSE, $2\mathcal{L}$, after each epoch.

2.2 Activation Functions

The logistic, hyperbolic-tangent, and linear activations are

$$f_{\log}(z) = \frac{1}{1 + e^{-z}}, \quad f'_{\log}(z) = f_{\log}(z)(1 - f_{\log}(z)), \quad (2.4)$$

$$f_{\tanh}(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad f'_{\tanh}(z) = 1 - \tanh^2(z), \quad (2.5)$$

$$f_{\text{lin}}(z) = z, \quad f'_{\text{lin}}(z) = 1. \quad (2.6)$$

Logistic outputs are thresholded at 0.5, and tanh outputs are thresholded at 0. The linear output is used for regression [3]. The code uses targets 0 and 1 for both classification activations. This is different from the usual tanh targets of -1 and $+1$.

2.3 Prediction and Evaluation

For three-class classification, one binary neuron is fitted to each pair of classes : $(0, 1)$, $(0, 2)$, and $(1, 2)$. Each model casts one vote, and the class with the largest vote count is returned. Ties are resolved deterministically by sorted class order. This one-against-one construction reduces the multiclass task to three binary problems [2]. The split is performed independently within each class to keep the classes balanced.

For class k , precision, recall, and the F1 score are

$$P_k = \frac{TP_k}{TP_k + FP_k}, \quad R_k = \frac{TP_k}{TP_k + FN_k}, \quad F_{1,k} = \frac{2P_k R_k}{P_k + R_k}. \quad (2.7)$$

The F1 score is the harmonic mean of precision and recall [4]. Macro precision, recall, and F1 are calculated for each class separately and then averaged. Therefore, every class has equal importance, regardless of its number of samples. Micro metrics first add the true positives, false positives, and false negatives from all classes and then calculate one overall score. Therefore, classes with more samples have a greater effect on the micro score. For this balanced, single-label classification task, micro precision, micro recall, and

micro F1 are equal to the overall accuracy. The code returns zero when a metric has a zero denominator. Regression performance is measured by

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}, \quad \% \text{RMSE} = 100 \frac{\text{RMSE}}{|\bar{y}|}, \quad |\bar{y}| > 0. \quad (2.8)$$

RMSE measures prediction error in the same units as the target [5]. The code divides percentage RMSE by the absolute target mean. This value is undefined when $\bar{y} = 0$, but neither regression split has a zero mean.

Chapter 3

Experimental Setup

The linearly separable and non-linearly separable classification datasets each contain three classes with 500 two-dimensional samples per class. A stratified 70:30 split gives 1050 training samples and 450 test samples for each dataset. Each test class has 150 samples. The regression datasets use the same 70:30 split. A random seed of 42 makes the splits repeatable. The supplied datasets do not include feature names or units, and the features are not standardized.

Classification uses learning rates 0.001, 0.01, 0.1, and 0.2 with 500, 1000, 1500, and 2000 epochs. Across two datasets and two activations, this produces 64 trained multiclass models. Regression uses learning rates 0.01 and 0.05 with 500 and 1000 epochs, producing eight models across the two datasets. The figures use the first best setting in ascending learning-rate and epoch order. The same test split is used to compare settings and report scores, so the word “best” refers only to this split.

NumPy performs the numerical work [6], and Matplotlib creates the plots [7]. The neuron, training method, data split, voting method, and metrics are implemented without a machine-learning library.

Chapter 4

Results

4.1 Classification Results

4.1.1 Hyperparameter Comparison

Table 4.1 reports test accuracy for every recorded setting. The linearly separable dataset reaches almost perfect accuracy for most settings. The non-linearly separable dataset stays near 83% because the model can only produce straight boundaries.

Table 4.1: Classification test accuracy (%) for the complete sweep.

Dataset	LR	Logistic: epochs				Tanh: epochs			
		500	1000	1500	2000	500	1000	1500	2000
Linearly separable	0.001	98.89	99.11	99.33	99.33	99.56	99.56	99.56	99.56
	0.01	99.56	99.56	99.56	99.56	99.78	100.00	100.00	100.00
	0.1	99.56	99.78	100.00	100.00	100.00	100.00	100.00	100.00
	0.2	99.78	100.00	100.00	100.00	99.56	100.00	100.00	100.00
Non-linearly separable	0.001	75.78	76.67	77.11	77.11	80.89	79.56	79.33	80.22
	0.01	79.33	80.00	79.33	79.56	80.89	81.78	82.00	82.22
	0.1	80.22	80.67	81.11	81.78	82.67	82.67	82.67	82.67
	0.2	80.67	81.78	82.22	82.22	82.67	82.67	82.67	82.67

4.1.2 Inferences from the Classification Sweep

Table 4.2: Accuracy range and number of tied best settings.

Dataset	Activation	Minimum (%)	Maximum (%)	Best settings
Linearly separable	Logistic	98.89	100.00	5 of 16
Linearly separable	Tanh	99.56	100.00	10 of 16
Non-linearly separable	Logistic	75.78	82.22	2 of 16
Non-linearly separable	Tanh	79.33	82.67	8 of 16

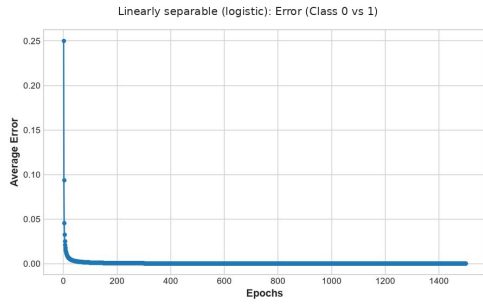
All linearly separable settings give 98.89–100.00% accuracy. Tanh has the smaller range and reaches 100.00% in 10 settings, compared with five for logistic. At matching settings, tanh equals or exceeds logistic in 15 of 16 cases. The exception is $\text{LR} = 0.2$ at 500 epochs: logistic gives 99.78% and tanh gives 99.56%. Tanh reaches 100.00% at $\text{LR} = 0.1$ after 500 epochs; logistic needs 1000 epochs at $\text{LR} = 0.2$. Higher learning rates mainly reduce the epochs needed to reach the same ceiling, after which extra epochs make no difference.

Logistic reaches its maximum only at $\text{LR} = 0.2$ with 1500 or 2000 epochs. Tanh reaches its maximum for all eight settings at $\text{LR} = 0.1$ or 0.2, so it has already plateaued by 500 epochs at these rates.

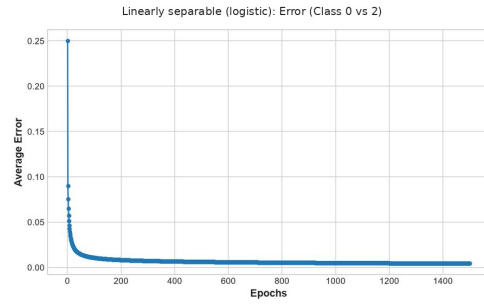
More epochs do not always improve test accuracy. Logistic at $\text{LR} = 0.01$ falls from 80.00% at 1000 epochs to 79.33% at 1500 epochs. Tanh at $\text{LR} = 0.001$ falls from 80.89% at 500 epochs to 79.33% at 1500 epochs, then rises to 80.22% at 2000 epochs. The best activation results differ by only 0.44 points, or two of the 450 test samples, so this is not proof that tanh is always better. The more important result is the 17.33-point gap between the best linearly and non-linearly separable accuracies, showing the limit of straight boundaries.

4.1.3 Linearly Separable Dataset

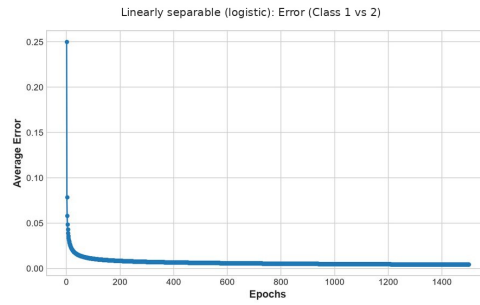
Logistic activation: LR = 0.1, 1500 epochs. The losses fall smoothly, and test accuracy reaches 100.00%.



(a) Error: C0–C1



(b) Error: C0–C2



(c) Error: C1–C2

Figure 4.1: Linearly separable logistic loss curves.

Logistic decision regions and test results.

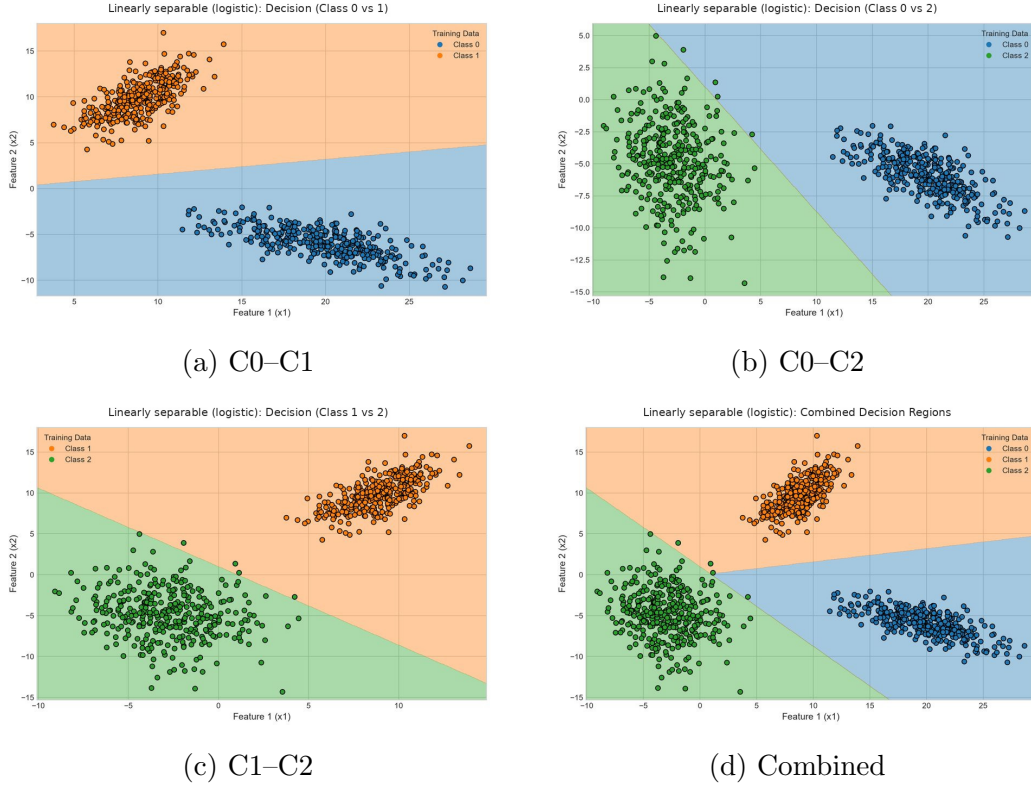


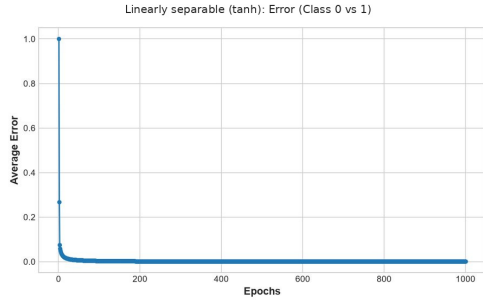
Figure 4.2: Linearly separable logistic decision regions.

Table 4.3: Linearly separable logistic confusion matrix and class-wise test metrics.

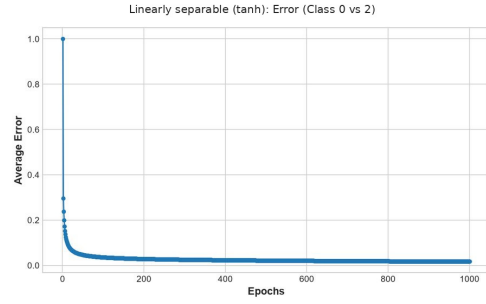
True	C0	C1	C2	Class	Precision	Recall	F1
C0	150	0	0	C0	100.00	100.00	100.00
C1	0	150	0	C1	100.00	100.00	100.00
C2	0	0	150	C2	100.00	100.00	100.00

Accuracy, macro F1, and micro F1: 100.00%.

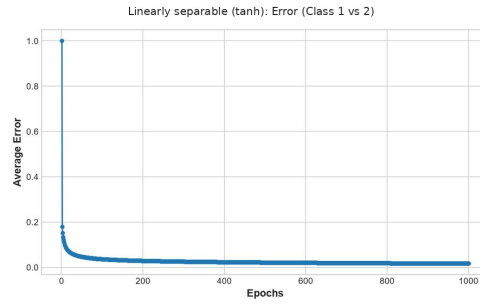
Tanh activation: LR = 0.01, 1000 epochs. The losses flatten within 1000 epochs, and test accuracy reaches 100.00%.



(a) Error: C0–C1



(b) Error: C0–C2



(c) Error: C1–C2

Figure 4.3: Linearly separable tanh loss curves.

Tanh decision regions and test results.

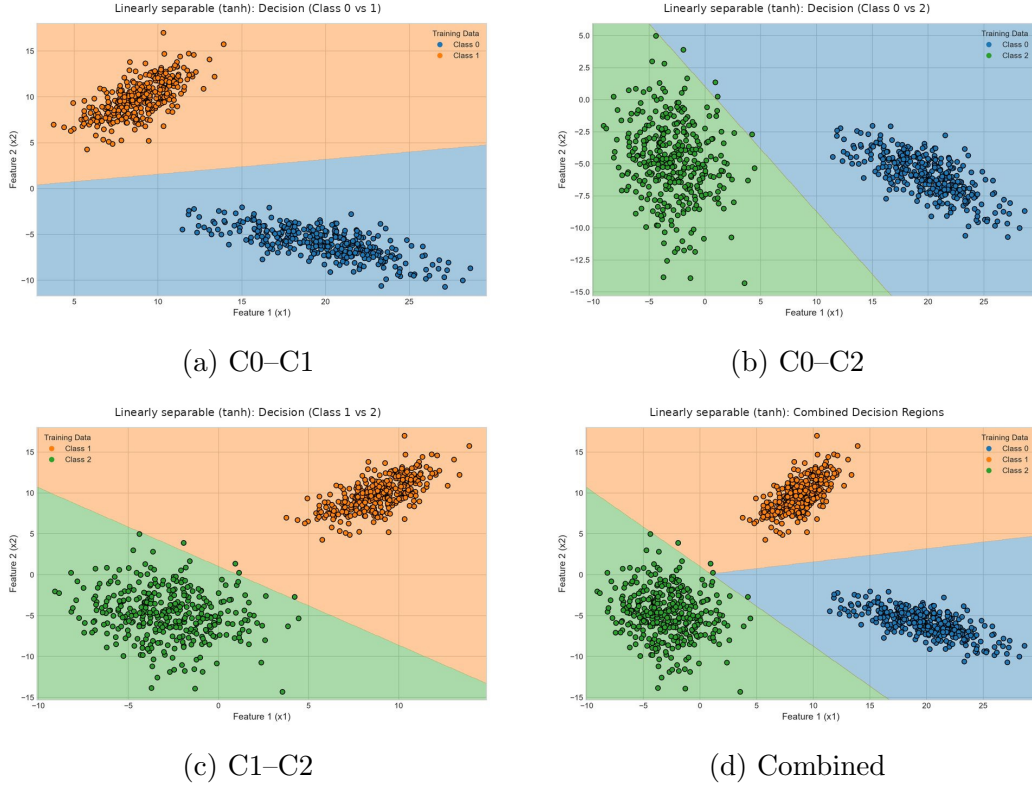


Figure 4.4: Linearly separable tanh decision regions.

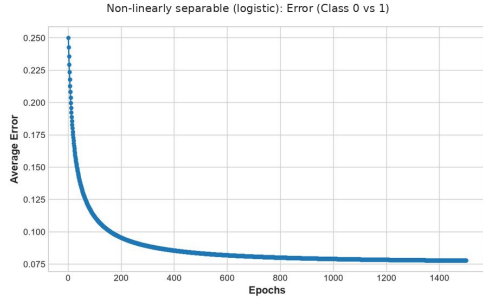
Table 4.4: Linearly separable tanh confusion matrix and class-wise test metrics.

True	C0	C1	C2	Class	Precision	Recall	F1
C0	150	0	0	C0	100.00	100.00	100.00
C1	0	150	0	C1	100.00	100.00	100.00
C2	0	0	150	C2	100.00	100.00	100.00

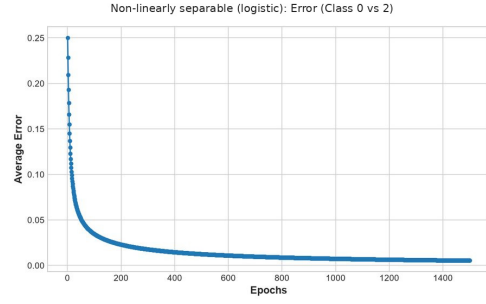
Accuracy, macro F1, and micro F1: 100.00%.

4.1.4 Non-linearly Separable Dataset

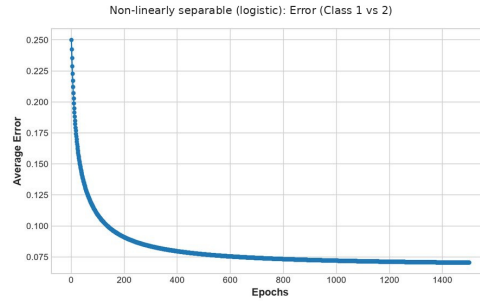
Logistic activation: LR = 0.2, 1500 epochs. The losses fall smoothly, but the straight boundaries limit test accuracy to 82.22%.



(a) Error: C0–C1



(b) Error: C0–C2



(c) Error: C1–C2

Figure 4.5: Non-linearly separable logistic loss curves.

Logistic decision regions and test results.

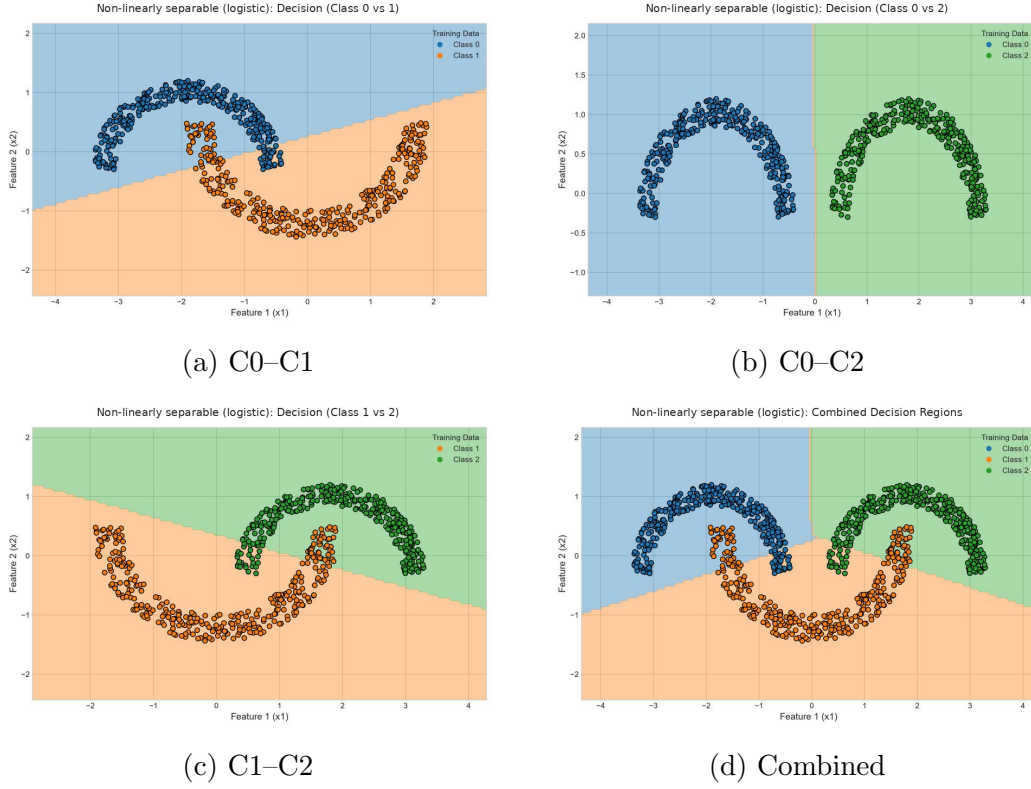


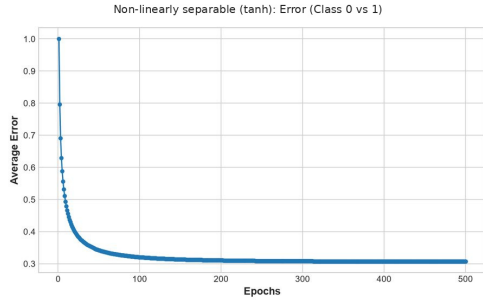
Figure 4.6: Non-linearly separable logistic decision regions.

Table 4.5: Non-linearly separable logistic confusion matrix and class-wise test metrics.

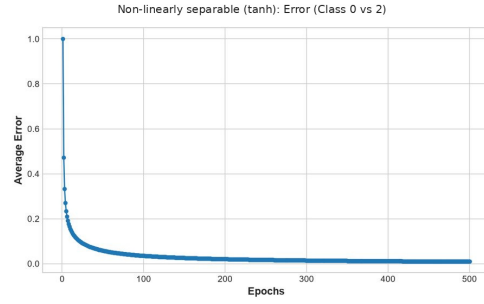
True	C0	C1	C2	Class	Precision	Recall	F1
C0	133	17	0	C0	86.93	88.67	87.79
C1	20	109	21	C1	73.65	72.67	73.15
C2	0	22	128	C2	85.91	85.33	85.62

Accuracy: 82.22%; macro F1: 82.19%; micro F1: 82.22%.

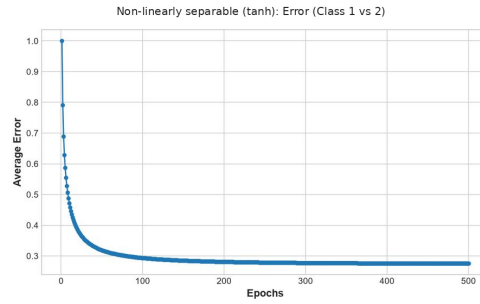
Tanh activation: LR = 0.1, 500 epochs. The losses flatten within 500 epochs, and test accuracy reaches 82.67%.



(a) Error: C0–C1



(b) Error: C0–C2



(c) Error: C1–C2

Figure 4.7: Non-linearly separable tanh loss curves.

Tanh decision regions and test results.

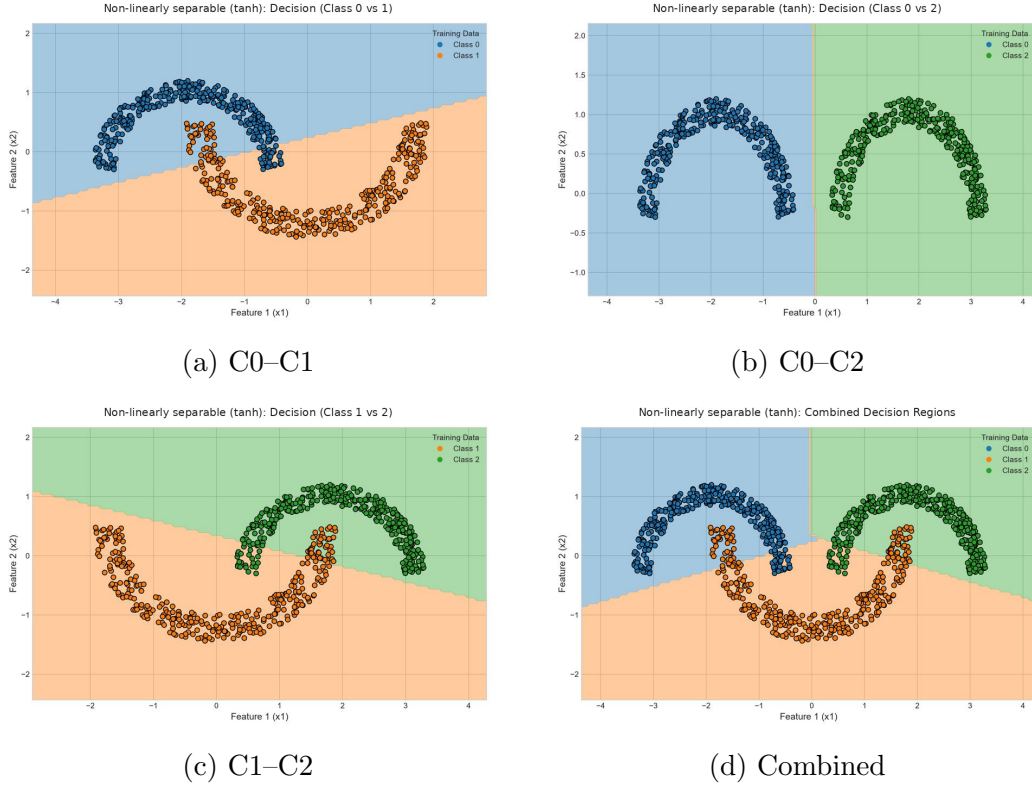


Figure 4.8: Non-linearly separable tanh decision regions.

Table 4.6: Non-linearly separable tanh confusion matrix and class-wise test metrics.

True	C0	C1	C2	Class	Precision	Recall	F1
C0	134	16	0	C0	87.58	89.33	88.45
C1	19	110	21	C1	74.32	73.33	73.83
C2	0	22	128	C2	85.91	85.33	85.62

Accuracy: 82.67%; macro F1: 82.63%; micro F1: 82.67%.

4.1.5 Classification Comparison

The main difference is the shape of the data. Straight boundaries separate the linearly separable classes, so both models reach 100.00% accuracy. The non-linearly separable classes are curved and overlap, so logistic makes 80 errors and tanh makes 78 errors. More epochs cannot make a single-neuron boundary curved.

Class 1 is the hardest non-linearly separable class. Its recall is 72.67% for logistic and 73.33% for tanh because it lies between the other two classes. The selected tanh settings use fewer epochs, but the fixed split and 0/1 tanh targets do not support a general claim that tanh is always better.

4.2 Regression Results

4.2.1 Hyperparameter Comparison

Table 4.7: Regression results for every recorded setting.

Dataset	Setting	Train RMSE	Train %RMSE	Test RMSE	Test %RMSE
Univariate	LR 0.01, 500 epochs	0.3533	16.30	0.3489	16.24
	LR 0.01, 1000 epochs	0.2736	12.62	0.2716	12.64
	LR 0.05, 500 epochs	0.1709	7.88	0.1727	8.04
	LR 0.05, 1000 epochs	0.1493	6.89	0.1528	7.11
Bivariate	LR 0.01, 500 epochs	1.5416	19.33	1.5309	19.15
	LR 0.01, 1000 epochs	1.4767	18.52	1.4709	18.40
	LR 0.05, 500 epochs	1.4271	17.90	1.4282	17.87
	LR 0.05, 1000 epochs	1.4218	17.83	1.4252	17.83

For univariate regression, $LR = 0.05$ gives a much larger improvement than $LR = 0.01$. At 500 epochs, increasing the learning rate lowers test RMSE from 0.3489 to 0.1727, a 50.50% reduction. At 1000 epochs, it lowers RMSE from 0.2716 to 0.1528, a 43.74% reduction. Doubling the training time from 500 to 1000 epochs lowers RMSE by 22.16% at $LR = 0.01$ and by 11.52% at $LR = 0.05$. Across the full univariate sweep, test RMSE falls by 56.21%

from the worst setting to the best setting. Thus, within the tested values, the learning rate has a larger effect than the extra 500 epochs.

The bivariate results change less. Increasing the learning rate lowers test RMSE by 6.71% at 500 epochs and by 3.11% at 1000 epochs. Doubling the epoch count lowers RMSE by 3.92% at $LR = 0.01$, but by only 0.21% at $LR = 0.05$. The best and worst bivariate settings differ by 6.90%, and the results at $LR = 0.05$ are almost unchanged between 500 and 1000 epochs. This indicates that training has nearly reached a plateau and that more epochs alone are unlikely to remove the remaining error.

The best percentage RMSE is 7.11% for the univariate data and 17.83% for the bivariate data. The bivariate value is about 2.5 times the univariate value, so the linear model gives a weaker relative fit to the bivariate data. Absolute RMSE values should not be compared directly because the two datasets may use different target scales. Percentage RMSE should also be interpreted with care because the implementation divides by the absolute mean target.

4.2.2 Univariate Regression

The selected model uses $LR = 0.05$ and 1000 epochs. Training RMSE is 0.1493 and test RMSE is 0.1528; the corresponding percentage values are 6.89% and 7.11%.

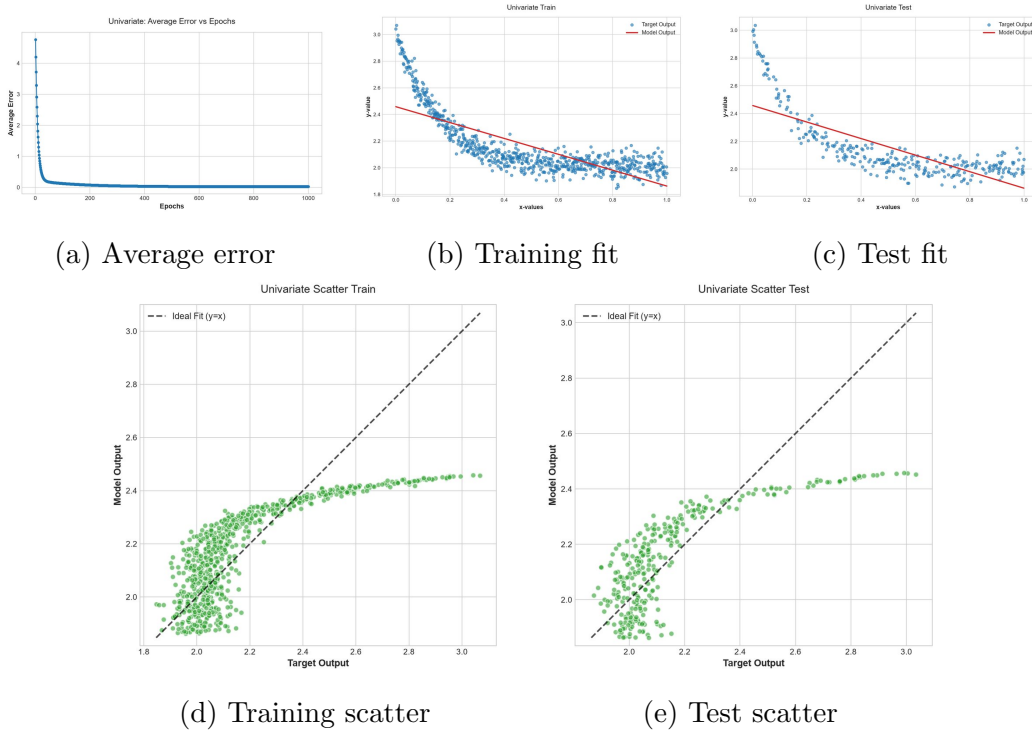


Figure 4.9: Univariate regression plots for $LR = 0.05$, 1000 epochs.

The fitted line captures the overall downward trend but misses the curve, especially at small input values. This also appears in the scatter plots. Train and test errors are close, and the learning curve is almost flat by the end.

4.2.3 Bivariate Regression

The selected model uses $LR = 0.05$ and 1000 epochs. Its training RMSE is 1.4218, its test RMSE is 1.4252, and its test percentage RMSE is 17.83%.

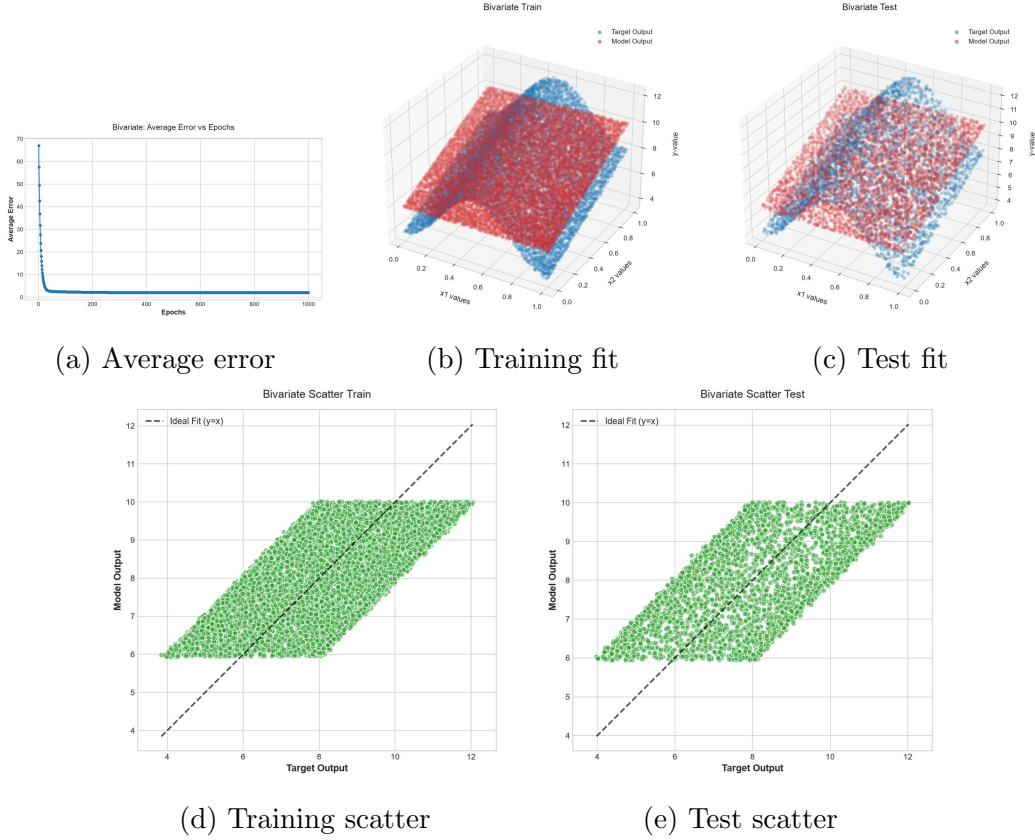


Figure 4.10: Bivariate regression plots for $LR = 0.05$, 1000 epochs.

Train and test errors are almost equal. However, the scatter is wide because one plane does not fully match the target surface. The 500- and 1000-epoch results are also nearly equal, so more training gives little improvement.

Chapter 5

Conclusion

The models were implemented without machine-learning libraries. Every linearly separable setting gives at least 98.89% accuracy, and several settings reach 100.00%. The highest non-linearly separable accuracy is 82.67%. Tanh is more stable across the tested classification settings and is better than logistic at every matching non-linearly separable setting. However, the two best non-linearly separable results differ by only two test samples. The larger 17.33-point gap between the two datasets comes from the straight decision boundaries of a single neuron. The tanh results also use 0/1 targets, so they are not a general proof that tanh is better than logistic activation.

For regression, the learning-rate change produces the largest improvement. The univariate RMSE falls by 56.21% from the worst setting to the best setting, while the bivariate reduction is only 6.90%. At $LR = 0.05$, another 500 epochs reduce bivariate RMSE by only 0.21%, showing a clear plateau. The best percentage RMSE is 7.11% for the univariate data and 17.83% for the bivariate data. A single line therefore fits the univariate data better than a single plane fits the bivariate data.

Future work can use a validation set, several random splits, standardized features, nonlinear features, hidden layers, and cross-entropy loss.

References

- [1] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958, doi: 10.1037/h0042519.
- [2] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
- [3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [4] C. J. van Rijsbergen, *Information Retrieval*, 2nd ed. London, U.K.: Butterworths, 1979.
- [5] T. Chai and R. R. Draxler, “Root mean square error (RMSE) or mean absolute error (MAE)?—Arguments against avoiding RMSE in the literature,” *Geoscientific Model Development*, vol. 7, no. 3, pp. 1247–1250, 2014, doi: 10.5194/gmd-7-1247-2014.
- [6] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.
- [7] J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007, doi: 10.1109/MCSE.2007.55.